Compte rendu TP3 - Interface Streamlit de MaintAI

Étudiante : Salma Riad
Groupe : à renseigner
Date : 10 novembre 2026
Module : Développement et déploiement d'applications intelligentes - séance 6

Objectif

Développer une interface Python Streamlit à deux onglets pour explorer des relevés de capteurs puis obtenir une prédiction de panne en appelant l'API FastAPI sécurisée par clé API.

1. Mise en place

```
pip install -r requirements.txt
python -m maintai.dataset
python -m maintai.train
python -m pytest
```

Le polycopié annonce **5 failed, 18 passed** sur le starter : les trois tests du client HTTP et les deux tests d'interface échouent tant que les TODO ne sont pas implémentés. Il s'agit du résultat initial attendu par le sujet, pas d'une nouvelle mesure après correction.

2. Démarrer Streamlit

```
streamlit run ui/app.py
python -m streamlit run ui/app.py
```

Le premier mode peut produire ModuleNotFoundError: No module named 'maintai' si le dossier racine n'est pas visible dans le chemin d'import Python. Le second exécute Streamlit comme module de l'environnement actif et permet de trouver le package. L'application s'ouvre normalement à http://localhost:8501.

3. Sprint 1 - Onglet Données

Le code affiche le nombre de relevés, la proportion des pannes et la température moyenne. Il calcule ensuite le taux de panne par tranche d'une variable choisie, et représente la température face à la vibration avec une couleur selon la présence d'une panne.

Extrait de `ui/app.py` :

```
c1, c2, c3 = st.columns(3)
c1.metric("Relevés", str(len(df)))
c2.metric("Taux de panne", f"{df.failure.mean():.1%}")
c3.metric("Température moyenne", f"{df.temperature_c.mean():.1f} °C")

variable = st.selectbox("Variable", FEATURES, index=1)
tranches = pd.cut(df[variable], bins=10)
taux_par_tranche = df.groupby(tranches, observed=True).failure.mean()
taux_par_tranche.index = [round(i.mid, 2) for i in taux_par_tranche.index]
st.bar_chart(taux_par_tranche)

points = df.assign(panne=df.failure.map({0: "Non", 1: "Oui"}))
st.scatter_chart(points, x="temperature_c", y="vibration_mm_s", color="panne")
```

Résultats calculés sur les 5 000 relevés synthétiques générés :

Indicateur	Valeur
Relevés	5 000
Taux de panne	13,6 %
Température moyenne	59,8 °C

Pour vibration_mm_s, les taux observés augmentent avec la vibration : environ 2,0 % sous 2,24 mm/s, 32,3 % entre 4,46 et 6,68 mm/s et 66,4 % entre 6,68 et 8,90 mm/s. Les tranches extrêmes contiennent peu de relevés. La vibration semble importante, mais l'influence des variables doit aussi être examinée conjointement.

Relances et cache. Chaque déplacement de widget relance normalement le script : un print("relance") placé au début serait donc exécuté à nouveau. @st.cache_data évite de relire le CSV à chaque interaction si les entrées n'ont pas changé. Les curseurs d'un st.form sont regroupés jusqu'au clic sur « Prédire ».

4. Sprint 2 - Onglet Prédiction

La méthode MaintaiClient.predict envoie les valeurs au point d'accès /v1/predict en JSON et transmet la clé via X-API-Key :

```
def predict(self, reading: dict) -> dict:
    headers = {"X-API-Key": self.api_key} if self.api_key else {}
    response = self.http.post("/v1/predict", json=reading, headers=headers)
    if response.status_code != 200:
        raise APIError(response.status_code, response.json().get("detail"))
    return response.json()
```

Dans ui/app.py, APIError produit un avertissement, httpx.HTTPError un message « API injoignable », et une réponse réussie affiche un indicateur de probabilité suivi d'un message « ALERTE » ou « Situation normale ».

Essai	Comportement prévu par le code
Relevé par défaut (72, 6, 85, 15000), bonne clé	Probabilité d'environ **95 %**, message **ALERTE** avec le modèle entraîné
Fausse clé	**Erreur 401 : Clé API absente ou invalide**
API arrêtée	**API injoignable**

L'API se lance avec uvicorn maintai.api.main:app --reload --env-file .env dans un premier terminal, puis Streamlit dans un second. Les libellés ci-dessus sont déduits du code et du modèle ; la démonstration visuelle dans le navigateur reste à effectuer sur le PC du groupe.

Tests finaux réellement exécutés :

```
.......................                                                  [100%]
23 passed
```

La suite teste l'interface par AppTest, le client avec un TestClient, ainsi que les cas 401 et 422.

Pourquoi CORS ne bloque pas le port 8501 ? Dans le TP2, JavaScript dans le navigateur envoyait directement une requête à FastAPI : le navigateur appliquait CORS. Avec Streamlit, la requête httpx part du

processus Python côté serveur, pas d'un script JavaScript interorigine. Le navigateur n'applique pas sa politique CORS à cet appel serveur à serveur.

5. Questions de compréhension

1. **Qui envoie les requêtes ?** Dans le TP2, le navigateur exécute le JavaScript. Dans le TP3, le processus Python de Streamlit envoie la requête à travers MaintaiClient.
2. **Pourquoi pas d'Uvicorn dans `test_client.py` ?** Le test injecte un client HTTP de test qui exécute les routes FastAPI en mémoire, sans serveur externe.
3. **Quelle interface choisir ?** Streamlit est rapide pour une démonstration avec graphiques. Pour un outil de maintenance pérenne exigeant une ergonomie très personnalisée, un front-end HTML/JavaScript est plus souple, mais demande davantage de développement.

6. Livraison Git

- Le fichier .gitignore exclut .env, data/*.csv et models/.
- **Pull request TP3 :** lien à compléter après publication sur GitHub.
- **Tag `tp3` :** à créer après fusion.
- **`git status` :** à vérifier dans le dépôt du groupe ; le ZIP transmis ne contient pas d'historique Git.

7. Bilan

Les deux onglets et l'appel de l'API sont implémentés et les **23 tests passent**. Restent la vérification visuelle sur le PC Windows et les opérations Git demandées par le professeur.